Lab Code [15 points]
Filename: chip.sv
AndrewID: ishikhar

```
`default_nettype none
//AI (codex 4ish messages) used to clean up the code (style things)
//AI (codex 5ish messages) used to rsync to ece before starting git
//AI (codex 2 messages) used to write duplicated blocks
// (deterministic things like copy paste this line of code for these cases)


// Uncomment exactly one complete ChipInterface module, including its header.
// Part 3 is selected.

// Part 1
// module ChipInterface (
//     output logic [6:0] DS1_SEG, DS2_SEG,
//     output logic DS1_DP, DS1_AN, DS2_DP, DS2_AN,
//     output logic [17:0] LD,
//     output logic [2:0] RGB0, RGB1,
//     input logic [17:0] SW,
//     input logic [3:0] BTN
// );
//     logic clock, reset, win;
//     logic [3:0] hMove, fMove;
//
//     assign clock = BTN[3];
//     assign reset = SW[17];
//     assign hMove = SW[15:12];
//
//     myExplicitFSM fsm (.fMove, .win, .q0(), .q1(), .q2(),
//                        .hMove, .clock, .reset);
//
//     HextoSevenSegment fsm_move_display (.hex(fMove), .segment(DS2_SEG));
//
//     assign DS1_SEG = 7'b1111111;
//     assign DS1_DP = 1'b1;
//     assign DS1_AN = 1'b1;
//     assign DS2_DP = 1'b1;
//     assign DS2_AN = 1'b0;
//     assign LD = '0;
//     assign RGB0 = {1'b0, win, 1'b0};
//     assign RGB1 = '0;
// endmodule : ChipInterface

// Part 2
// module ChipInterface (
//     output logic [6:0] DS1_SEG, DS2_SEG,
//     output logic DS1_DP, DS1_AN, DS2_DP, DS2_AN,
//     output logic [17:0] LD,
//     output logic [2:0] RGB0, RGB1,
//     input logic [17:0] SW,
//     input logic [3:0] BTN
// );
//     logic clock, reset, win;
//     logic [3:0] hMove, fMove;
//
//     assign clock = BTN[3];
//     assign reset = SW[17];
//     assign hMove = SW[15:12];
//
//     myAbstractFSM fsm (.fMove, .win, .hMove, .clock, .reset);
//
//     HextoSevenSegment fsm_move_display (.hex(fMove), .segment(DS2_SEG));
//
//     assign DS1_SEG = 7'b1111111;
//     assign DS1_DP = 1'b1;
//     assign DS1_AN = 1'b1;
//     assign DS2_DP = 1'b1;
//     assign DS2_AN = 1'b0;
//     assign LD = '0;
//     assign RGB0 = {1'b0, win, 1'b0};
//     assign RGB1 = '0;
```

```
// endmodule : ChipInterface

// Part 3
module ChipInterface (
    output logic [3:0] D1_AN, D2_AN,
    output logic [7:0] D1_SEG, D2_SEG,
    output logic [17:0] LD,
    output logic [2:0] RGB0, RGB1,
    input logic [17:0] SW,
    input logic [3:0] BTN,
    input logic CLOCK_100
);
    logic [3:0] hMove, fMove;
    logic clock, enter, new_game, reset, win;
    logic [3:0] HEX7, HEX6, HEX5, HEX4, HEX3, HEX2, HEX1, HEX0;
    logic [7:0] blank;

    (* ASYNC_REG = "TRUE" *) logic [6:0] inputs_meta, inputs_sync;
    always_ff @(posedge CLOCK_100) begin
        inputs_meta <= {SW[15], BTN[0], BTN[3], SW[3:0]};
        inputs_sync <= inputs_meta;
    end

    assign clock = CLOCK_100;
    assign {reset, new_game, enter, hMove} = inputs_sync;
    myExpandedFSM fsm (
        .fMove, .win, .HEX7, .HEX6, .HEX5, .HEX4,
        .HEX3, .HEX2, .HEX1, .HEX0, .blank,
        .hMove, .enter, .new_game, .clock, .reset
    );

    EightSevenSegmentDisplays displays (
        .HEX7, .HEX6, .HEX5, .HEX4, .HEX3, .HEX2, .HEX1, .HEX0,
        .CLOCK_100, .reset, .blank, .dec_points(8'b0),
        .D1_AN, .D2_AN, .D1_SEG, .D2_SEG
    );

    assign LD = {14'b0, fMove};
    assign RGB0 = 3'b000;
    assign RGB1 = win ? 3'b101 : 3'b000;
endmodule : ChipInterface
```

Lab Code [15 points]
Filename: part1.sv
AndrewID: ishikhar

```
`default_nettype none
module dFlipFlop (
    output logic q,
    input logic d, clock, reset
);
    always_ff @(posedge clock)
        if (reset)
            q <= 1'b0;
        else
            q <= d;
endmodule : dFlipFlop

module myExplicitFSM (
    output logic [3:0] fMove,
    output logic win, q0, q1, q2,
    input logic [3:0] hMove,
    input logic clock, reset
);
    logic d0, d1, d2;
    logic START5, PLAY1, PLAY3, WIN9, WIN7, WIN2;
    logic h2, h3, h4, h6, h7, h8, h9;
    logic p1Win9, p1Invalid, p3Win2, p3Invalid;
    logic nextPLAY1, nextPLAY3, nextWIN9, nextWIN7, nextWIN2;

    dFlipFlop ff0 (.d(d0), .q(q0), .*);
    dFlipFlop ff1 (.d(d1), .q(q1), .*);
    dFlipFlop ff2 (.d(d2), .q(q2), .*);

    assign START5 = ~q2 & ~q1 & ~q0;
    assign PLAY1  = ~q2 & ~q1 &  q0;
    assign PLAY3  = ~q2 &  q1 & ~q0;
    assign WIN9   =  q2 & ~q1 & ~q0;
    assign WIN7   =  q2 & ~q1 &  q0;
    assign WIN2   =  q2 &  q1 & ~q0;

    assign h2 = (hMove == 4'h2);
    assign h3 = (hMove == 4'h3);
    assign h4 = (hMove == 4'h4);
    assign h6 = (hMove == 4'h6);
    assign h7 = (hMove == 4'h7);
    assign h8 = (hMove == 4'h8);
    assign h9 = (hMove == 4'h9);

    assign p1Win9 = h2 | h3 | h4 | h7 | h8;
    assign p1Invalid = ~(p1Win9 | h9);
    assign p3Win2 = h4 | h7 | h8;
    assign p3Invalid = ~(p3Win2 | h2);

    assign nextPLAY1 = (START5 & h6) | (PLAY1 & p1Invalid);
    assign nextPLAY3 = (PLAY1 & h9) | (PLAY3 & p3Invalid);
    assign nextWIN9 = (PLAY1 & p1Win9) | WIN9;
    assign nextWIN7 = (PLAY3 & h2) | WIN7;
    assign nextWIN2 = (PLAY3 & p3Win2) | WIN2;

    assign d0 = nextPLAY1 | nextWIN7;
    assign d1 = nextPLAY3 | nextWIN2;
    assign d2 = nextWIN9 | nextWIN7 | nextWIN2;

    assign fMove[3] = WIN9;
    assign fMove[2] = START5 | WIN7;
    assign fMove[1] = PLAY3 | WIN7 | WIN2;
    assign fMove[0] = START5 | PLAY1 | PLAY3 | WIN9 | WIN7;
    assign win = WIN9 | WIN7 | WIN2;
endmodule : myExplicitFSM

//module part1_tb;
//    logic [3:0] fMove;
//    logic win;
//    logic q2,q1,q0;
```

```
//    logic [3:0] hMove;
//    logic clock, reset;
//
//    myExplicitFSM dut (
//        .fMove(fMove),
//        .win(win),
//        .q0(q0),
//        .q1(q1),
//        .q2(q2),
//        .hMove(hMove),
//        .clock(clock),
//        .reset(reset)
//    );
//
//    logic START5, PLAY1, PLAY3, WIN9, WIN7, WIN2;
//    logic [2:0] state_bits;
//    assign state_bits = {q2,q1,q0};
//
//    initial begin
//        clock = 0;
//        forever #5 clock = ~clock;
//    end
//    assign START5 = ~q2 & ~q1 & ~q0;
//    assign PLAY1  = ~q2 & ~q1 &  q0;
//    assign PLAY3  = ~q2 &  q1 & ~q0;
//    assign WIN9   =  q2 & ~q1 & ~q0;
//    assign WIN7   =  q2 & ~q1 &  q0;
//    assign WIN2   =  q2 &  q1 & ~q0;
//
//    initial begin
//        $monitor($time,, "state=%b, fMove = %d, hMove = %d, win = %b",
//                 state_bits, fMove, hMove, win);
//        hMove = 4'hF;
//        reset = 1'b1;
//        @(posedge clock); #1;
//        if (!START5) $display("Expected START5 after reset");
//        reset = 1'b0;
//
//        // START5 loops on an invalid move
//        hMove = 4'hF;
//        @(posedge clock); #1;
//        if (!START5) $display("Expected START5 after invalid move");
//
//        hMove <= 4'h3;
//        @(posedge clock); #1
//        if (!START5) $display("Expected START5 after incorrect move");
//
//        hMove = 4'h6;
//        @(posedge clock); #1;
//        if (!PLAY1) $display("Expected PLAY1 after hMove = 6");
//
//        // PLAY1 loops on invalid moves and changes to PLAY3 on 9.
//        hMove = 4'h5;
//        @(posedge clock); #1;
//        if (!PLAY1) $display("Expected PLAY1 after invalid move");
//
//        hMove = 4'h9;
//        @(posedge clock); #1;
//        if (!PLAY3) $display("Expected PLAY3 after hMove = 9");
//
//        // PLAY3 loops on invalid moves and can end in WIN7.
//        hMove = 4'h5;
//        @(posedge clock); #1;
//        if (!PLAY3) $display("Expected PLAY3 after invalid move");
//
//        hMove = 4'h2;
//        @(posedge clock); #1;
//        if (!WIN7 || !win) $display("Expected WIN7 after hMove = 2");
//
//        hMove = 4'hF;
//        @(posedge clock); #1;
```

```
141 //         if (!WIN7 || !win) $display("Expected WIN7 to hold");
142 //
143 //         // Reset and test PLAY1 -> WIN9, then its self-loop.
144 //         reset = 1'b1;
145 //         @(posedge clock); #1;
146 //         reset = 1'b0;
147 //         hMove = 4'h6;
148 //         @(posedge clock); #1;
149 //         hMove = 4'h3;
150 //         @(posedge clock); #1;
151 //         if (!WIN9 || !win) $display("Expected WIN9 after hMove = 3");
152 //
153 //         hMove = 4'hF;
154 //         @(posedge clock); #1;
155 //         if (!WIN9 || !win) $display("Expected WIN9 to hold");
156 //
157 //         // Reset and test PLAY3 -> WIN2, then its self-loop.
158 //         reset = 1'b1;
159 //         @(posedge clock); #1;
160 //         reset = 1'b0;
161 //         hMove = 4'h6;
162 //         @(posedge clock); #1;
163 //         hMove = 4'h9;
164 //         @(posedge clock); #1;
165 //         hMove = 4'h4;
166 //         @(posedge clock); #1;
167 //         if (!WIN2 || !win) $display("Expected WIN2 after hMove = 4");
168 //
169 //         hMove = 4'hF;
170 //         @(posedge clock); #1;
171 //         if (!WIN2 || !win) $display("Expected WIN2 to hold");
172 //
173 //         // Reset from WIN2.
174 //         reset = 1'b1;
175 //         @(posedge clock); #1;
176 //         if (!START5) $display("Expected START5 after reset from WIN2");
177 //         reset = 1'b0;
178 //
179 //         // Reset from PLAY1.
180 //         hMove = 4'h6;
181 //         @(posedge clock); #1;
182 //         reset = 1'b1;
183 //         @(posedge clock); #1;
184 //         if (!START5) $display("Expected START5 after reset from PLAY1");
185 //         reset = 1'b0;
186 //
187 //         // Reset from PLAY3.
188 //         hMove = 4'h6;
189 //         @(posedge clock); #1;
190 //         hMove = 4'h9;
191 //         @(posedge clock); #1;
192 //         reset = 1'b1;
193 //         @(posedge clock); #1;
194 //         if (!START5) $display("Expected START5 after reset from PLAY3");
195 //         reset = 1'b0;
196 //
197 //         // Reset from WIN9.
198 //         hMove = 4'h6;
199 //         @(posedge clock); #1;
200 //         hMove = 4'h3;
201 //         @(posedge clock); #1;
202 //         reset = 1'b1;
203 //         @(posedge clock); #1;
204 //         if (!START5) $display("Expected START5 after reset from WIN9");
205 //         reset = 1'b0;
206 //
207 //         // Reset from WIN7.
208 //         hMove = 4'h6;
209 //         @(posedge clock); #1;
210 //         hMove = 4'h9;
211 //         @(posedge clock); #1;
```

```
212 //        hMove = 4'h2;
213 //        @(posedge clock); #1;
214 //        reset = 1'b1;
215 //        @(posedge clock); #1;
216 //        if (!START5) $display("Expected START5 after reset from WIN7");
217 //        reset = 1'b0;
218 //
219 //        $display("PASS: part1_tb");
220 //        $finish;
221 //    end
222 //endmodule : part1_tb
```

Lab Code [15 points]
Filename: part2.sv
AndrewID: ishikhar

```
`default_nettype none
module myAbstractFSM (
    output logic [3:0] fMove,
    output logic win,
    input logic [3:0] hMove,
    input logic clock, reset
);
    enum logic [2:0] {START5, PLAY1, PLAY3, WIN9, WIN7, WIN2} currState;
    enum logic [2:0] nextState;

    always_comb begin
        case (currState)
            START5:
                if (hMove == 4'h6)
                    nextState = PLAY1;
                else
                    nextState = START5;
            PLAY1:
                if (hMove == 4'h9)
                    nextState = PLAY3;
                else if (hMove == 4'h2 || hMove == 4'h3 || hMove == 4'h4 ||
                         hMove == 4'h7 || hMove == 4'h8)
                    nextState = WIN9;
                else
                    nextState = PLAY1;
            PLAY3:
                if (hMove == 4'h2)
                    nextState = WIN7;
                else if (hMove == 4'h4 || hMove == 4'h7 || hMove == 4'h8)
                    nextState = WIN2;
                else
                    nextState = PLAY3;
            WIN9: nextState = WIN9;
            WIN7: nextState = WIN7;
            WIN2: nextState = WIN2;
            default: nextState = START5;
        endcase
    end

    always_comb begin
        fMove = 4'b0000;
        win = 1'b0;
        unique case (currState)
            START5: fMove = 4'b0101;
            PLAY1: fMove = 4'b0001;
            PLAY3: fMove = 4'b0011;
            WIN9: begin
                fMove = 4'b1001;
                win = 1'b1;
            end
            WIN7: begin
                fMove = 4'b0111;
                win = 1'b1;
            end
            WIN2: begin
                fMove = 4'b0010;
                win = 1'b1;
            end
            default: begin
                fMove = 4'b0000;
                win = 1'b0;
            end
        endcase
    end

    always_ff @(posedge clock)
        if (reset)
            currState <= START5;
        else
```

```
 70             currState <= nextState;
 71 endmodule : myAbstractFSM
 72
 73 //module part2_tb;
 74 //    logic [3:0] fMove;
 75 //    logic win;
 76 //    logic [3:0] hMove;
 77 //    logic clock, reset;
 78 //
 79 //    myAbstractFSM dut (
 80 //        .fMove(fMove),
 81 //        .win(win),
 82 //        .hMove(hMove),
 83 //        .clock(clock),
 84 //        .reset(reset)
 85 //    );
 86 //
 87 //
 88 //    initial begin
 89 //        clock = 0;
 90 //        forever #5 clock = ~clock;
 91 //    end
 92 //
 93 //    initial begin
 94 //        $monitor($time,, "state=%0d, fMove = %d, hMove = %d, win = %b",
 95 //                  dut.currState, fMove, hMove, win);
 96 //        hMove = 4'hF;
 97 //        reset = 1'b1;
 98 //        @(posedge clock); #1;
 99 //        if (dut.currState != dut.START5)
100 //            $display("Expected START5 after reset");
101 //        reset = 1'b0;
102 //
103 //        // START5 loops on an invalid move
104 //        hMove = 4'hF;
105 //        @(posedge clock); #1;
106 //        if (dut.currState != dut.START5)
107 //            $display("Expected START5 after invalid move");
108 //
109 //        hMove <= 4'h3;
110 //        @(posedge clock); #1
111 //        if (dut.currState != dut.START5)
112 //            $display("Expected START5 after incorrect move");
113 //
114 //        hMove = 4'h6;
115 //        @(posedge clock); #1;
116 //        if (dut.currState != dut.PLAY1)
117 //            $display("Expected PLAY1 after hMove = 6");
118 //
119 //        // PLAY1 loops on invalid moves and changes to PLAY3 on 9.
120 //        hMove = 4'h5;
121 //        @(posedge clock); #1;
122 //        if (dut.currState != dut.PLAY1)
123 //            $display("Expected PLAY1 after invalid move");
124 //
125 //        hMove = 4'h9;
126 //        @(posedge clock); #1;
127 //        if (dut.currState != dut.PLAY3)
128 //            $display("Expected PLAY3 after hMove = 9");
129 //
130 //        // PLAY3 loops on invalid moves and can end in WIN7.
131 //        hMove = 4'h5;
132 //        @(posedge clock); #1;
133 //        if (dut.currState != dut.PLAY3)
134 //            $display("Expected PLAY3 after invalid move");
135 //
136 //        hMove = 4'h2;
137 //        @(posedge clock); #1;
138 //        if (dut.currState != dut.WIN7 || !win)
139 //            $display("Expected WIN7 after hMove = 2");
140 //
```

```
141 //         hMove = 4'hF;
142 //         @(posedge clock); #1;
143 //         if (dut.currState != dut.WIN7 || !win)
144 //             $display("Expected WIN7 to hold");
145 //
146 //         // Reset and test PLAY1 -> WIN9, then its self-loop.
147 //         reset = 1'b1;
148 //         @(posedge clock); #1;
149 //         reset = 1'b0;
150 //         hMove = 4'h6;
151 //         @(posedge clock); #1;
152 //         hMove = 4'h3;
153 //         @(posedge clock); #1;
154 //         if (dut.currState != dut.WIN9 || !win)
155 //             $display("Expected WIN9 after hMove = 3");
156 //
157 //         hMove = 4'hF;
158 //         @(posedge clock); #1;
159 //         if (dut.currState != dut.WIN9 || !win)
160 //             $display("Expected WIN9 to hold");
161 //
162 //         // Reset and test PLAY3 -> WIN2, then its self-loop.
163 //         reset = 1'b1;
164 //         @(posedge clock); #1;
165 //         reset = 1'b0;
166 //         hMove = 4'h6;
167 //         @(posedge clock); #1;
168 //         hMove = 4'h9;
169 //         @(posedge clock); #1;
170 //         hMove = 4'h4;
171 //         @(posedge clock); #1;
172 //         if (dut.currState != dut.WIN2 || !win)
173 //             $display("Expected WIN2 after hMove = 4");
174 //
175 //         hMove = 4'hF;
176 //         @(posedge clock); #1;
177 //         if (dut.currState != dut.WIN2 || !win)
178 //             $display("Expected WIN2 to hold");
179 //
180 //         // Reset from WIN2.
181 //         reset = 1'b1;
182 //         @(posedge clock); #1;
183 //         if (dut.currState != dut.START5)
184 //             $display("Expected START5 after reset from WIN2");
185 //         reset = 1'b0;
186 //
187 //         // Reset from PLAY1.
188 //         hMove = 4'h6;
189 //         @(posedge clock); #1;
190 //         reset = 1'b1;
191 //         @(posedge clock); #1;
192 //         if (dut.currState != dut.START5)
193 //             $display("Expected START5 after reset from PLAY1");
194 //         reset = 1'b0;
195 //
196 //         // Reset from PLAY3.
197 //         hMove = 4'h6;
198 //         @(posedge clock); #1;
199 //         hMove = 4'h9;
200 //         @(posedge clock); #1;
201 //         reset = 1'b1;
202 //         @(posedge clock); #1;
203 //         if (dut.currState != dut.START5)
204 //             $display("Expected START5 after reset from PLAY3");
205 //         reset = 1'b0;
206 //
207 //         // Reset from WIN9.
208 //         hMove = 4'h6;
209 //         @(posedge clock); #1;
210 //         hMove = 4'h3;
211 //         @(posedge clock); #1;
```

```
212 //        reset = 1'b1;
213 //        @(posedge clock); #1;
214 //        if (dut.currState != dut.START5)
215 //            $display("Expected START5 after reset from WIN9");
216 //        reset = 1'b0;
217 //
218 //        // Reset from WIN7.
219 //        hMove = 4'h6;
220 //        @(posedge clock); #1;
221 //        hMove = 4'h9;
222 //        @(posedge clock); #1;
223 //        hMove = 4'h2;
224 //        @(posedge clock); #1;
225 //        reset = 1'b1;
226 //        @(posedge clock); #1;
227 //        if (dut.currState != dut.START5)
228 //            $display("Expected START5 after reset from WIN7");
229 //        reset = 1'b0;
230 //
231 //        $display("PASS: part2_tb");
232 //        $finish;
233 //    end
234 //endmodule : part2_tb
```

Lab Code [15 points]
Filename: part3.sv
AndrewID: ishikhar

```
`default_nettype none
module myExpandedFSM (
    output logic [3:0] fMove,
    output logic win,
    output logic [3:0] HEX7, HEX6, HEX5, HEX4,
    output logic [3:0] HEX3, HEX2, HEX1, HEX0,
    output logic [7:0] blank,
    input logic [3:0] hMove,
    input logic enter, new_game,
    input logic clock, reset
);
    typedef enum logic [5:0] {
        START5, HOLD6, PLAY1, HOLD9, PLAY3,
        HOLD_H2_WIN9, HOLD_H3_WIN9, HOLD_H4_WIN9,
        HOLD_H7_WIN9, HOLD_H8_WIN9,
        HOLD_H2_WIN7, HOLD_H4_WIN2, HOLD_H7_WIN2, HOLD_H8_WIN2,
        INVALID5, INVALID1, INVALID3,
        WIN9_H2, WIN9_H3, WIN9_H4, WIN9_H7, WIN9_H8,
        WIN7_H2, WIN2_H4, WIN2_H7, WIN2_H8,
        WIN9_H2_READY_NEW, WIN9_H2_WAIT_BOTH_RELEASE,
        WIN9_H3_READY_NEW, WIN9_H3_WAIT_BOTH_RELEASE,
        WIN9_H4_READY_NEW, WIN9_H4_WAIT_BOTH_RELEASE,
        WIN9_H7_READY_NEW, WIN9_H7_WAIT_BOTH_RELEASE,
        WIN9_H8_READY_NEW, WIN9_H8_WAIT_BOTH_RELEASE,
        WIN7_H2_READY_NEW, WIN7_H2_WAIT_BOTH_RELEASE,
        WIN2_H4_READY_NEW, WIN2_H4_WAIT_BOTH_RELEASE,
        WIN2_H7_READY_NEW, WIN2_H7_WAIT_BOTH_RELEASE,
        WIN2_H8_READY_NEW, WIN2_H8_WAIT_BOTH_RELEASE
    } state_t;
    state_t currState, nextState;
    always_comb begin
        nextState = currState;
        case (currState)
            START5:
                if (enter) begin
                    if (hMove == 4'd6) nextState = HOLD6;
                    else nextState = INVALID5;
                end
            HOLD6: if (!enter) nextState = PLAY1;
            PLAY1:
                if (enter) begin
                    case (hMove)
                        4'd2: nextState = HOLD_H2_WIN9;
                        4'd3: nextState = HOLD_H3_WIN9;
                        4'd4: nextState = HOLD_H4_WIN9;
                        4'd7: nextState = HOLD_H7_WIN9;
                        4'd8: nextState = HOLD_H8_WIN9;
                        4'd9: nextState = HOLD9;
                        default: nextState = INVALID1;
                    endcase
                end
            HOLD9: if (!enter) nextState = PLAY3;
            PLAY3:
                if (enter) begin
                    case (hMove)
                        4'd2: nextState = HOLD_H2_WIN7;
                        4'd4: nextState = HOLD_H4_WIN2;
                        4'd7: nextState = HOLD_H7_WIN2;
                        4'd8: nextState = HOLD_H8_WIN2;
                        default: nextState = INVALID3;
                    endcase
                end
            HOLD_H2_WIN9: if (!enter) nextState = WIN9_H2;
            HOLD_H3_WIN9: if (!enter) nextState = WIN9_H3;
            HOLD_H4_WIN9: if (!enter) nextState = WIN9_H4;
            HOLD_H7_WIN9: if (!enter) nextState = WIN9_H7;
            HOLD_H8_WIN9: if (!enter) nextState = WIN9_H8;
            HOLD_H2_WIN7: if (!enter) nextState = WIN7_H2;
            HOLD_H4_WIN2: if (!enter) nextState = WIN2_H4;
```

```
            HOLD_H7_WIN2: if (!enter) nextState = WIN2_H7;
            HOLD_H8_WIN2: if (!enter) nextState = WIN2_H8;
            INVALID5: if (!enter) nextState = START5;
            INVALID1: if (!enter) nextState = PLAY1;
            INVALID3: if (!enter) nextState = PLAY3;
            WIN9_H2:
                if (!new_game) nextState = WIN9_H2_READY_NEW;
            WIN9_H2_READY_NEW:
                if (new_game) nextState = WIN9_H2_WAIT_BOTH_RELEASE;
            WIN9_H2_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN9_H3:
                if (!new_game) nextState = WIN9_H3_READY_NEW;
            WIN9_H3_READY_NEW:
                if (new_game) nextState = WIN9_H3_WAIT_BOTH_RELEASE;
            WIN9_H3_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN9_H4:
                if (!new_game) nextState = WIN9_H4_READY_NEW;
            WIN9_H4_READY_NEW:
                if (new_game) nextState = WIN9_H4_WAIT_BOTH_RELEASE;
            WIN9_H4_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN9_H7:
                if (!new_game) nextState = WIN9_H7_READY_NEW;
            WIN9_H7_READY_NEW:
                if (new_game) nextState = WIN9_H7_WAIT_BOTH_RELEASE;
            WIN9_H7_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN9_H8:
                if (!new_game) nextState = WIN9_H8_READY_NEW;
            WIN9_H8_READY_NEW:
                if (new_game) nextState = WIN9_H8_WAIT_BOTH_RELEASE;
            WIN9_H8_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN7_H2:
                if (!new_game) nextState = WIN7_H2_READY_NEW;
            WIN7_H2_READY_NEW:
                if (new_game) nextState = WIN7_H2_WAIT_BOTH_RELEASE;
            WIN7_H2_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN2_H4:
                if (!new_game) nextState = WIN2_H4_READY_NEW;
            WIN2_H4_READY_NEW:
                if (new_game) nextState = WIN2_H4_WAIT_BOTH_RELEASE;
            WIN2_H4_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN2_H7:
                if (!new_game) nextState = WIN2_H7_READY_NEW;
            WIN2_H7_READY_NEW:
                if (new_game) nextState = WIN2_H7_WAIT_BOTH_RELEASE;
            WIN2_H7_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            WIN2_H8:
                if (!new_game) nextState = WIN2_H8_READY_NEW;
            WIN2_H8_READY_NEW:
                if (new_game) nextState = WIN2_H8_WAIT_BOTH_RELEASE;
            WIN2_H8_WAIT_BOTH_RELEASE:
                if (!new_game && !enter) nextState = START5;
            default: nextState = START5;
        endcase
    end
    always_comb begin
        fMove = 4'd5;
        win = 1'b0;
        {HEX7, HEX6, HEX5, HEX4} = 16'h0000;
        {HEX3, HEX2, HEX1, HEX0} = 16'h5000;
        blank = 8'b1111_0111;
        case (currState)
            START5, INVALID5: begin end
            HOLD6: begin
```

```
141                 {HEX7, HEX6, HEX5, HEX4} = 16'h6000;
142                 blank = 8'b0111_0111;
143             end
144             PLAY1, INVALID1: begin
145                 fMove = 4'd1;
146                 {HEX7, HEX6, HEX5, HEX4} = 16'h6000;
147                 {HEX3, HEX2, HEX1, HEX0} = 16'h1500;
148                 blank = 8'b0111_0011;
149             end
150             HOLD9, HOLD_H2_WIN9, HOLD_H3_WIN9, HOLD_H4_WIN9,
151             HOLD_H7_WIN9, HOLD_H8_WIN9: begin
152                 fMove = 4'd1;
153                 {HEX3, HEX2, HEX1, HEX0} = 16'h1500;
154                 blank = 8'b0011_0011;
155                 case (currState)
156                     HOLD_H2_WIN9: {HEX7, HEX6, HEX5, HEX4} = 16'h2600;
157                     HOLD_H3_WIN9: {HEX7, HEX6, HEX5, HEX4} = 16'h3600;
158                     HOLD_H4_WIN9: {HEX7, HEX6, HEX5, HEX4} = 16'h4600;
159                     HOLD_H7_WIN9: {HEX7, HEX6, HEX5, HEX4} = 16'h6700;
160                     HOLD_H8_WIN9: {HEX7, HEX6, HEX5, HEX4} = 16'h6800;
161                     default:      {HEX7, HEX6, HEX5, HEX4} = 16'h6900;
162                 endcase
163             end
164             PLAY3, INVALID3: begin
165                 fMove = 4'd3;
166                 {HEX7, HEX6, HEX5, HEX4} = 16'h6900;
167                 {HEX3, HEX2, HEX1, HEX0} = 16'h1350;
168                 blank = 8'b0011_0001;
169             end
170             HOLD_H2_WIN7, HOLD_H4_WIN2, HOLD_H7_WIN2, HOLD_H8_WIN2: begin
171                 fMove = 4'd3;
172                 {HEX3, HEX2, HEX1, HEX0} = 16'h1350;
173                 blank = 8'b0001_0001;
174                 case (currState)
175                     HOLD_H2_WIN7: {HEX7, HEX6, HEX5, HEX4} = 16'h2690;
176                     HOLD_H4_WIN2: {HEX7, HEX6, HEX5, HEX4} = 16'h4690;
177                     HOLD_H7_WIN2: {HEX7, HEX6, HEX5, HEX4} = 16'h6790;
178                     default:      {HEX7, HEX6, HEX5, HEX4} = 16'h6890;
179                 endcase
180             end
181             WIN9_H2, WIN9_H2_READY_NEW, WIN9_H2_WAIT_BOTH_RELEASE,
182             WIN9_H3, WIN9_H3_READY_NEW, WIN9_H3_WAIT_BOTH_RELEASE,
183             WIN9_H4, WIN9_H4_READY_NEW, WIN9_H4_WAIT_BOTH_RELEASE,
184             WIN9_H7,WIN9_H7_READY_NEW, WIN9_H7_WAIT_BOTH_RELEASE,
185             WIN9_H8, WIN9_H8_READY_NEW, WIN9_H8_WAIT_BOTH_RELEASE:
186             begin
187                 fMove = 4'd9;
188                 win = 1'b1;
189                 {HEX3, HEX2, HEX1, HEX0} = 16'h1590;
190                 blank = 8'b0011_0001;
191                 case (currState)
192                     WIN9_H2, WIN9_H2_READY_NEW, WIN9_H2_WAIT_BOTH_RELEASE:
193                         {HEX7, HEX6, HEX5, HEX4} = 16'h2600;
194                     WIN9_H3, WIN9_H3_READY_NEW, WIN9_H3_WAIT_BOTH_RELEASE:
195                         {HEX7, HEX6, HEX5, HEX4} = 16'h3600;
196                     WIN9_H4, WIN9_H4_READY_NEW, WIN9_H4_WAIT_BOTH_RELEASE:
197                         {HEX7, HEX6, HEX5, HEX4} = 16'h4600;
198                     WIN9_H7, WIN9_H7_READY_NEW, WIN9_H7_WAIT_BOTH_RELEASE:
199                         {HEX7, HEX6, HEX5, HEX4} = 16'h6700;
200                     default: {HEX7, HEX6, HEX5, HEX4} = 16'h6800;
201                 endcase
202             end
203             WIN7_H2, WIN7_H2_READY_NEW, WIN7_H2_WAIT_BOTH_RELEASE: begin
204                 fMove = 4'd7;
205                 win = 1'b1;
206                 {HEX7, HEX6, HEX5, HEX4} = 16'h2690;
207                 {HEX3, HEX2, HEX1, HEX0} = 16'h1357;
208                 blank = 8'b0001_0000;
209             end
210             WIN2_H4, WIN2_H4_READY_NEW, WIN2_H4_WAIT_BOTH_RELEASE, WIN2_H7,
211                 WIN2_H7_READY_NEW,
```

```
212             WIN2_H7_WAIT_BOTH_RELEASE, WIN2_H8, WIN2_H8_READY_NEW,
213                 WIN2_H8_WAIT_BOTH_RELEASE: begin
214                 fMove = 4'd2;
215                 win = 1'b1;
216                 {HEX3, HEX2, HEX1, HEX0} = 16'h1235;
217                 blank = 8'b0001_0000;
218                 case (currState)
219                     WIN2_H4, WIN2_H4_READY_NEW, WIN2_H4_WAIT_BOTH_RELEASE:
220                         {HEX7, HEX6, HEX5, HEX4} = 16'h4690;
221                     WIN2_H7, WIN2_H7_READY_NEW, WIN2_H7_WAIT_BOTH_RELEASE:
222                         {HEX7, HEX6, HEX5, HEX4} = 16'h6790;
223                     default: {HEX7, HEX6, HEX5, HEX4} = 16'h6890;
224                 endcase
225             end
226             default: begin end
227         endcase
228     end
229     always_ff @(posedge clock)
230         if (reset)
231             currState <= START5;
232         else
233             currState <= nextState;
234 endmodule : myExpandedFSM
```

Lab Code [15 points]
Filename: seven_segment.sv
AndrewID: ishikhar

```
1  // Required in every 18-240 file, but commented out to prevent synthesis issues
2  // `default_nettype none
3
4  /*
5   * This module should be interfaced with a combinational 7-Segment Display
6   * module in order to display 8 BCD values upon the Boolean board. It is
7   * intended to abstract away the sequential logic required to use the
8   * 7-Segment Displays present on the Boolean Board for 18-240 students.
9   *
10  * Date:   1/27/25
11  * Author: Rudy Sorensen (rsorense)
12  * 4 Feb 2025, WAN: Modified significantly for Lab 2
13  */
14
15 module EightSevenSegmentDisplays
16   (input  logic [3:0] HEX7, HEX6, HEX5, HEX4, HEX3, HEX2, HEX1, HEX0,
17    input  logic       CLOCK_100, reset,
18    input  logic [7:0] dec_points,
19    input  logic [7:0] blank,
20    output logic [3:0] D2_AN, D1_AN,
21    output logic [7:0] D2_SEG, D1_SEG);
22
23   logic [7:0] segments7, segments6, segments5, segments4,
24               segments3, segments2, segments1, segments0;
25
26    SevenSegmentDisplay ssd(.num7(HEX7),
27                            .num6(HEX6),
28                            .num5(HEX5),
29                            .num4(HEX4),
30                            .num3(HEX3),
31                            .num2(HEX2),
32                            .num1(HEX1),
33                            .num0(HEX0),
34                            .blank,
35                            .dec_points,
36                            .HEX7(segments7),
37                            .HEX6(segments6),
38                            .HEX5(segments5),
39                            .HEX4(segments4),
40                            .HEX3(segments3),
41                            .HEX2(segments2),
42                            .HEX1(segments1),
43                            .HEX0(segments0)
44                           );
45
46   logic cycle_cnt_clr;
47   logic index_cnt_en;
48   logic [1:0] anode_index;
49   logic [3:0] anode_index_onecold;
50   logic [16:0] cycle_cnt;
51   /*
52    * This counter reset value should be calculated by hand using the frequency
53    * of your source clock to achieve the target frequency of 250Hz to 10KHz.
54    *
55    * The lower limit to the frequency comes from the fact that the human eye
56    * de-saturates after around 20ms, while the upper limit is derived from the
57    * need for the display to be on for at least 100us so that the value can
58    * actually be perceived.
59    *
60    * Since the Boolean board has a 100MHz clock, we will count to 100,000. This
61    * means that each segment will be lit for 1ms and flashed again 3ms later
62    * because we are cycling through 4 values for each segment.
63    */
64
65   localparam CNT_LIMIT = 17'd100_000;
66
67   assign cycle_cnt_clr = cycle_cnt == CNT_LIMIT;
68   assign index_cnt_en = cycle_cnt_clr;
69
```

```
 70   SSDCounter #(17) cycle_cntr (
 71     .CLOCK_100,
 72     .reset,
 73     .clr(cycle_cnt_clr),
 74     .cnt(cycle_cnt),
 75     .en(1'b1)
 76     );
 77
 78   SSDCounter #(2) index_cntr (
 79     .CLOCK_100,
 80     .reset,
 81     .clr(1'b0),
 82     .cnt(anode_index),
 83     .en(index_cnt_en)
 84     );
 85
 86   OneColdDecoder #(4) anode_decode (
 87     .sel(anode_index),
 88     .out(anode_index_onecold)
 89     );
 90
 91   always_comb begin
 92     D1_AN = anode_index_onecold;
 93     D2_AN = anode_index_onecold;
 94
 95     case (anode_index)
 96       2'd0: begin
 97         D2_SEG = segments0;
 98         D1_SEG = segments4;
 99       end
100       2'd1: begin
101         D2_SEG = segments1;
102         D1_SEG = segments5;
103       end
104       2'd2: begin
105         D2_SEG = segments2;
106         D1_SEG = segments6;
107       end
108       2'd3: begin
109         D2_SEG = segments3;
110         D1_SEG = segments7;
111       end
112     endcase
113
114   end
115
116 endmodule : EightSevenSegmentDisplays
117
118 module OneColdDecoder #(
119   parameter NUM_OUTPUTS = 4
120   ) (
121   input  logic [$clog2(NUM_OUTPUTS)-1:0] sel,
122   output logic [NUM_OUTPUTS-1:0] out
123   );
124
125   always_comb begin
126     out = {NUM_OUTPUTS{1'b1}};
127     out[sel] = 1'b0;
128   end
129
130 endmodule : OneColdDecoder
131
132 module SSDCounter #(
133   parameter WIDTH = 8
134   ) (
135   input  logic CLOCK_100,
136   input  logic reset,
137   input  logic clr,
138   input  logic en,
139   output logic [WIDTH-1:0] cnt
140   );
```

```
141
142   always_ff @(posedge CLOCK_100, posedge reset) begin
143     if (reset)
144       cnt <= '0;
145     else if (clr)
146       cnt <= '0;
147     else if (en)
148       cnt <= cnt + 1'b1;
149   end
150
151 endmodule : SSDCounter
152
153 /*
154  *  This is essentially the SevenSegmentDisplay from HW2. There are
155  *  two differences.  Firstly, it correctly decodes hexadecimal values
156  *  and not just BCD values.  Secondly, the dec_points input allows
157  *  the decimal points on the 8 displays to be illuminated.
158  */
159 module SevenSegmentDisplay
160   (input logic [3:0] num7, num6, num5, num4, num3, num2, num1, num0,
161    input logic [7:0] blank,
162    input logic [7:0] dec_points,
163    output logic [7:0] HEX7, HEX6, HEX5, HEX4, HEX3, HEX2, HEX1, HEX0);
164
165    logic [6:0] seg7, seg6, seg5, seg4, seg3, seg2, seg1, seg0;
166
167    HextoSevenSegment digit0(.hex(num0), .segment(seg0));
168    HextoSevenSegment digit1(.hex(num1), .segment(seg1));
169    HextoSevenSegment digit2(.hex(num2), .segment(seg2));
170    HextoSevenSegment digit3(.hex(num3), .segment(seg3));
171    HextoSevenSegment digit4(.hex(num4), .segment(seg4));
172    HextoSevenSegment digit5(.hex(num5), .segment(seg5));
173    HextoSevenSegment digit6(.hex(num6), .segment(seg6));
174    HextoSevenSegment digit7(.hex(num7), .segment(seg7));
175
176    assign HEX0 = (blank[0]) ? '1 : {~dec_points[0], seg0};
177    assign HEX1 = (blank[1]) ? '1 : {~dec_points[1], seg1};
178    assign HEX2 = (blank[2]) ? '1 : {~dec_points[2], seg2};
179    assign HEX3 = (blank[3]) ? '1 : {~dec_points[3], seg3};
180    assign HEX4 = (blank[4]) ? '1 : {~dec_points[4], seg4};
181    assign HEX5 = (blank[5]) ? '1 : {~dec_points[5], seg5};
182    assign HEX6 = (blank[6]) ? '1 : {~dec_points[6], seg6};
183    assign HEX7 = (blank[7]) ? '1 : {~dec_points[7], seg7};
184
185 endmodule: SevenSegmentDisplay
186
187 /*
188  * Very similar to the BCDtoSevenSegment module from HW2, but
189  * now with hex support!
190  */
191 module HextoSevenSegment
192   (input  logic [3:0] hex,
193    output logic [6:0] segment);
194
195   always_comb
196     unique case (hex)
197       4'h0: segment = 7'b1000000;
198       4'h1: segment = 7'b1111001;
199       4'h2: segment = 7'b0100100;
200       4'h3: segment = 7'b0110000;
201       4'h4: segment = 7'b0011001;
202       4'h5: segment = 7'b0010010;
203       4'h6: segment = 7'b0000010;
204       4'h7: segment = 7'b1111000;
205       4'h8: segment = 7'b0000000;
206       4'h9: segment = 7'b0011000;
207       4'hA: segment = 7'b0001000;
208       4'hB: segment = 7'b0000011;
209       4'hC: segment = 7'b1000110;
210       4'hD: segment = 7'b0100001;
211       4'hE: segment = 7'b0000110;
```

```
212       4'hF: segment = 7'b0001110;
213     endcase
214
215 endmodule : HextoSevenSegment
```